

HK221

THI CUỐI KỲ



TRƯỜNG ĐH BÁCH KHOA – ĐHQG-HCM
KHOA ĐIỆN ĐIỆN TỬ

Môn học

Hệ thống điều khiển nhúng

Mã môn học

EE3067

Thời lượng

80 phút

Mã đề

Học kỳ/năm học 1 2022-2023
Ngày thi 21/12/2022

Ghi chú: - Được sử dụng tài liệu

Câu 1: (L.O.1) (2,5đ)

Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (8-bit địa chỉ A7-A0; 16-bit dữ liệu D15-D0; RD, WR trên 2 chân riêng biệt) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không giải mã địa chỉ các kênh):

- 1/ Module 1: 16 kênh ADC 16-bit
- 2/ Module 2: 10 kênh DAC 12-bit
- 3/ Module 3: 16 kênh PWM 10-bit

- 4/ Module 4: 16 kênh Encoder 32-bit
- 5/ Module 5: 8 kênh Digital Input 8-bit
- 6/ Module 6: 4 kênh Digital Output 8-bit

Câu 2: (L.O.2) (2,5đ)

Viết chương trình Verilog thực hiện xuất tín hiệu xung với tần số thay đổi như sau:

- Tên module: xuất_xung (clk, PulseOut, mode)
- 2 đầu vào:
 - o tín hiệu xung clock 1us: clk.
 - o mode: chọn tần số xuất xung (2 bits)
 - mode = 0: xuất xung tần số 1 KHz
 - mode = 1: xuất xung tần số 0.2 KHz
 - mode = 2: xuất xung tần số 0.5 KHz
 - mode = 3: xuất xung tần số 0.1 Hz;
- Tín hiệu đầu ra: PulseOut.

Câu 3: (L.O.5) (2,5đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port {PD8, PD9} cho UART, {PE5, PE6} cho PWM, và {PB5, PB6} cho CAN. Thạch anh sử dụng cho STM32F4 là 25MHz.

- a. Cấu hình các thanh ghi RCC để cho phép CPU chạy ở tần số 168MHz.
- b. Cấu hình các thanh ghi hệ thống để sử dụng 3 chức năng UART, PWM, CAN và UART có khả năng hoạt động DMA.

Câu 4: (L.O.6) (2,5đ)

Trong cấu hình ở Câu 1, giả sử dùng STM32F4 để giải mã địa chỉ 16-bit dữ liệu và địa chỉ bắt đầu của Module 3 CS_PWM là 0x6000_0040. Hàm nhận dữ liệu từ UART được cho trước như sau:

- Hàm nhận N byte từ bộ đệm rxbuff qua UART: void receive_data (char *rxbuff, int N). Biết dữ liệu rxbuff dạng binary với tổng số byte nhận là 35 và format chuỗi dữ liệu nhận vào:

0xAA (1byte)	0x55 (1byte)	PWM1 (byteH)	PWM1 (byteL)	---	PWM16 (byteH)	PWM16 (byteL)	CRC (1byte)
-----------------	-----------------	-----------------	-----------------	-----	------------------	------------------	----------------

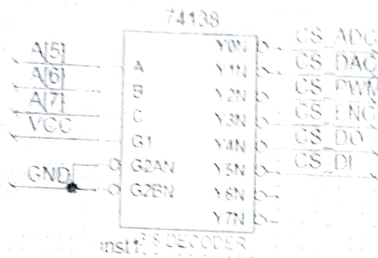
Viết chương trình đọc dữ liệu từ UART, kiểm tra chuỗi dữ liệu nếu header bằng {0xAA, 0x55} và đúng CRC thì chuyển đổi giá trị và xuất ra 16 kênh PWM, biết rằng CRC bằng XOR 34 byte dữ liệu trong chuỗi (từ byte thứ 1 đến byte thứ 34).

ĐÁP ÁN

Câu 1: (2,5đ)

- Vẽ đúng sơ đồ giải mã 74138

(0,5đ)



- Ghi đúng tầm địa chỉ cho 6 module

(1đ)

- Vẽ đúng sơ đồ kết nối 6 module

(1đ)

- Chú ý: Giải mã cho 16-bit data D[15:0].

Nếu sinh viên làm giải mã 8-bit data D[7:0] sẽ mất toàn bộ điểm câu 1

Câu 2: (2,5đ)

- Khai báo đúng module, đầu vào, đầu ra

(0,5đ)

- Xuất đúng tần số cho 2 giá trị của mode

(2đ)

Câu 3: (2,5đ)

a. (1,0đ)

- Cho phép PLL 168MHz:

$RCC_CR |= (1 << 16) | (1 << 24);$

(1đ)

$RCC_PLLCFGR |= (25 << 0) | (336 << 6) | (2 << 16)$

//cho phép HSE, PLL

//M=25, N = 336, P = 2

$$25 / M * N / P = 168$$

b. (1,5đ)

- Cấu hình thành ghi xung clock

(0,5đ)

$RCC_AHB1ENR |= (1 << 1) | (1 << 3) | (1 << 4) | (1 << 21);$ //clock cho PB, PD, PE, DMA1

$RCC_APB1ENR |= (1 << 18) | (1 << 26);$ //clock cho UART3, CAN2

$RCC_APB2ENR |= (1 << 16);$ //clock cho TIM9

- Cấu hình mode chức năng

(1đ)

$GPIOB_MODER |= (2 << 10) | (2 << 12);$

$GPIOD_MODER |= (2 << 16) | (2 << 18);$

$GPIOE_MODER |= (2 << 10) | (2 << 12);$

$GPIOB_AFRL |= (9 << 20) | (9 << 24);$

$GPIOD_AFRH |= (7 << 0) | (7 << 4);$

$GPIOE_AFRL |= (3 << 20) | (3 << 24);$

$DMA_S1CR |= (4 << 25);$

//DMA1, Stream1, Channel4, UART3_RX

$DMA_S3CR |= (4 << 25);$

//DMA1, Stream3, Channel4, UART3_TX

Câu 4: (2,5đ)

```
char rxbuff[35];
uint16_t *p;
uint8_t crc, i;

main(){
    while(1){
        receive_data(rxbuff, 35);
        crc = rxbuff[0];
        for(i=1; i<34; i++)    crc ^= rxbuff[i];
        if ((rxbuff[0]==0xAA) && (rxbuff[1]==0x55) && (rxbuff[34]==crc))
        {
            p = 0x60000040;
            for(i=1; i<17; i++)
                *p++ = (uint16_t)rxbuff[2*i]*256 + rxbuff[2*i+1];
        }
    }
}
```